P2 | AI MODEL TRAINING REPORT

호텔 예약 취소 예측
인공지능 학습 결과서

5 개 모델 사전 설계 · 모델링 및 성능 비교 · 최종 Stacking 선정

최종 선정 모델 | Stacking

XGBoost + Random Forest + MLP → Logistic Regression 메타모델

Accuracy	Precision	Recall	F1-score	ROC-AUC
0.8658	0.8493	0.7755	0.8108	0.9402

확장 분석 포함: Stacking 의 혼동행렬·임계값 운영 시나리오·예약 특성별 오류 분석 설계·저장 모델 재현성 점검. 확인되지 않은 세그먼트 지표와 재로딩 결과는 “미검증”으로 명시했다.

Executive Summary

본 프로젝트는 예약 취소 여부(is_canceled)를 예측하기 위해 Decision Tree, Logistic Regression, Random Forest, XGBoost, Stacking 총 5 가지 모델을 처음부터 후보로 설계하고 모델링 결과를 비교하였다. 팀 최종 비교표는 공통 데이터 사용을 전제로 작성했지만, 저장된 개별 노트북에서는 일부 분할 생성 방식과 실행 데이터 건수에 차이가 확인된다.

팀 최종 비교표에서 Stacking 모델은 Accuracy 와 F1-score 가 가장 높고, XGBoost 보다 Recall 도 소폭 높았다. ROC-AUC 는 XGBoost 가 근소하게 우수하지만 취소 탐지 성능의 균형을 고려해 Stacking 모델을 선정했다. 다만 일부 노트북은 데이터 분할 및 실행 버전이 달라, 작은 점수 차이의 통계적 우위는 아직 입증되지 않았다.

보고서의 핵심 흐름 단일 모델 개발 후 스태킹을 추가한 과정이 아니라, 5 개 후보를 초기 설계하고 동일한 문제에 대한 모델별 결과를 비교하여 최종 모델을 선택한 프로젝트이다.

보고서 구성

분석 단계	내용
01–02	분석 목표 · 공통 데이터/검증 설계 · 사전 후보 5 개
03	5 개 모델 최종 성능 비교 및 Stacking 선정
04–07	최종 Stacking 의 OOF 구조 · 튜닝 · 과적합 점검 · 특성 중요도
08–12	선정 근거 · Threshold/오류 · 세그먼트 분석 · 재현성 · 결론

작성 자료: 팀 최종 모델 성능 비교표, 5 개 ipynb, 모델별 개인 학습 결과서, Stacking 실험 메모(notion.pdf).

01 분석 목표와 공통 데이터 설계

동일한 호텔 예약 취소 예측 문제에 대해 다섯 후보를 동등한 비교 대상으로 사전 설정했다.

1.1 이진 분류 문제 정의

타깃 변수는 is_canceled 이며 예약 유지(0)와 취소(1)를 구분한다. 모델의 확률 예측을 기반으로 취소 위험 예약을 선제적으로 식별하는 것이 목적이다. 취소를 놓치는 FN 과 불필요한 위험 경고 FP 의 균형을 위해 F1-score 를 중요한 선택 기준으로 삼고, Accuracy · Recall · Precision · ROC-AUC 도 함께 비교하였다.

1.2 데이터 규모 및 공통 분할

데이터 단계	건수	비고
전체 예약 데이터	119,208 건	공통 전처리 결과
팀 비교표 기준 Train / Test	95,367 / 23,841 건	5 개 모델 평가용으로 취합한 기준 건수 (동일 행 재검증 필요)

팀이 계획한 비교 기준은 공통 전처리 자료와 Train/Test 분할을 사용하는 것이다. 그러나 보관된 Random Forest 노트북은 원본 RAW 파일에서 전처리와 그룹 분할을 자체 생성했으며, StackingB.ipynb 는 저장된 split 을 읽었지만 팀 집계보다 Train 1 건이 적고 Test 1 건이 많았다. 따라서 아래 공통 건수는 팀 비교표 기준이며, 모든 실행본의 실제 평가 행이 완전히 동일하다는 검증 결과는 아니다.

1.3 학습 검증 및 과적합 평가 설계

모델	검증 방법	주요 확인 지표
Decision Tree	5-Fold (노트북 cv=5)	Train/Test F1, CV F1
Logistic Regression	StratifiedGroupKFold 5	Train/CV F1, Test F1
Random Forest	StratifiedGroupKFold 5	Train/CV ROC-AUC
XGBoost	StratifiedGroupKFold 5	CV F1, Test 성능
Stacking	내부 Train 그룹 5-Fold + OOF / 별도 내부 Validation	OOF Train(참고), Validation, Test

Stacking 검증 방식 Stacking 모델은 그룹 기반 5-Fold 교차검증으로 OOF 확률을 생성하여 메타모델을 학습한다. 이후 Train 내부 Validation 과 공통 Test 에서 결과를 확인한다.

1.4 전처리 보고서와의 역할 분리

결측치 · 이상치 · 누수 변수 제거 · 원핫인코딩 등 전체 전처리 상세는 별도 전처리 보고서에서 다룬다. 본 문서는 전처리 재설명보다 모델 학습 · 비교와 최종 Stacking 검증에 집중한다.

기록 유의: 팀 비교표 기준 Train 95,367 건 / Test 23,841 건과 달리 StackingB.ipynb 저장 실행본은 Train 95,366 건 / Test 23,842 건이다. Random Forest 도 원본에서 자체 분할을 생성했으므로, 재현 가능한 공통 평가 집합의 확정이 필요하다.

02 5 개 후보 모델의 설계 및 학습

다섯 모델을 처음부터 비교 후보로 설정하고 모델별 최적화를 거쳐 테스트 성능을 취합했다.

후보 모델	선정 및 학습 목적	학습·개선 방법
Decision Tree	단일 트리 기반의 해석 가능한 비선형 기준선	깊이 및 분할·리프 크기 제어 / Grid·Randomized 탐색
Logistic Regression	선형 판단 기준과 규제 효과 검증	StandardScaler, L1, class_weight / Grid·Randomized
Random Forest	Bagging 으로 분산을 낮춘 트리 앙상블	트리 깊이·리프 최소 표본 크기 / 그룹 5-Fold
XGBoost	Boosting 기반 비선형 관계 학습	트리 개수·학습률·깊이 / Grid·Randomized
Stacking	서로 다른 예측 패턴의 3 개 모델을 결합	XGBoost·RF·MLP + OOF / 로지스틱 메타모델

2.1 단일 모델 4 종의 주요 실험 관찰

- Decision Tree: 기본 모델 Train F1 0.9929 vs Test F1 0.7565 로 과적합이 컸다. RandomizedSearchCV 에서는 격차가 0.0129 로 줄었고, 최종 비교에는 Test F1 이 더 높은 GridSearchCV(클래스 1 F1 0.7744)를 반영했다.
- Logistic Regression: L1 규제(C≈0.114), class_weight=balanced, 로그 변환과 임계값 0.45 를 적용해 Test F1 0.7614 를 기록했다.
- Random Forest: 튜닝 이후 Precision 이 0.9609 까지 높아졌지만 Recall 0.4498 로 낮아 취소 탐지 균형이 약해졌다.
- XGBoost: 탐색 후 Accuracy 0.8644, F1 0.8078, ROC-AUC 0.9407 로 강력한 단일 모델 기준선을 확보했다.

2.2 Stacking 의 초기 설계 목적

단일 트리·선형·Bagging·Boosting 모델과 함께 Stacking 모델을 독립된 다섯 번째 후보로 설계했다. Stacking 모델은 XGBoost, Random Forest, MLP 세 기초 모델이 생성한 취소 확률을 메타모델 Logistic Regression 이 통합하는 구조다. 서로 다른 모델이 제공하는 정보가 상호 보완되는지 최종 비교에서 검증하도록 설계했다.

2.3 비교의 전제

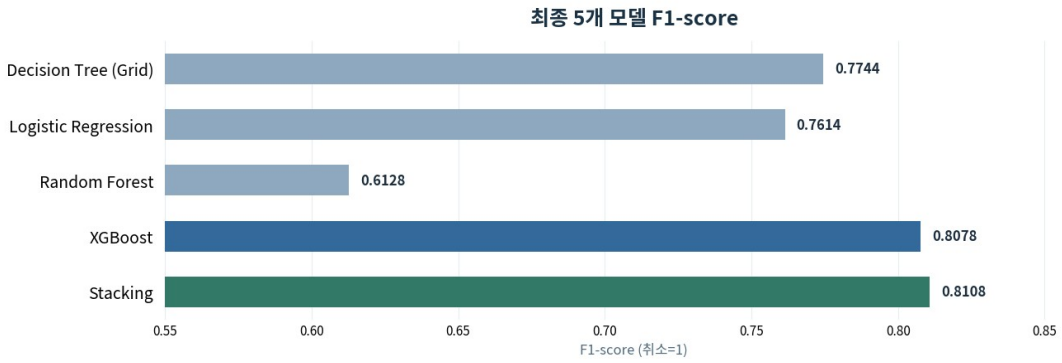
다섯 후보는 동일한 타깃을 예측했고 팀 최종 성능 비교표에 결과를 취합했다. 다만 개별 실행본에서는 Random Forest 가 자체 그룹 분할을 생성했으며 Stacking 의 저장된 split 집계도 팀 기준과 1 건 다르다. 전처리 Pipeline·튜닝 지표·임계값에도 차이가 있으므로, 아래 점수는 팀의 모델 선정 기록이지 완전히 동일한 실험 조건에서 검증한 순수 모델 효과를 뜻하지 않는다. Decision Tree 의 비교값은 RandomizedSearchCV 가 아닌 GridSearchCV 의 테스트 결과를 채택했다.

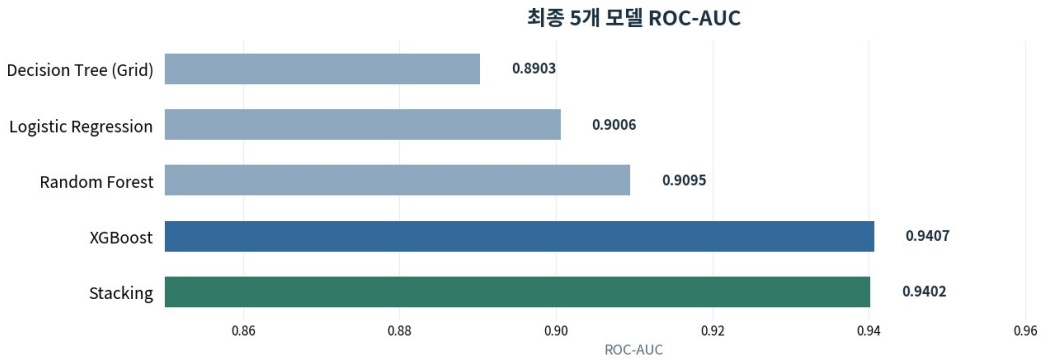
03 다섯 모델의 성능 비교와 선정

다섯 후보의 테스트 결과를 비교한 뒤 최종 모델을 선정했다. Decision Tree 는 GridSearchCV 의 취소(1) 평가 수치를 기준으로 정리했다.

모델	Accuracy	Precision	Recall	F1-score	ROC-AUC
Decision Tree (Grid)	0.8377	0.7990	0.7513	0.7744	0.8903
Logistic Regression	0.8086	0.7080	0.8234	0.7614	0.9006
Random Forest	0.7892	0.9609	0.4498	0.6128	0.9095
XGBoost	0.8644	0.8514	0.7685	0.8078	0.9407
Stacking	0.8658	0.8493	0.7755	0.8108	0.9402

표 1. 5 개 후보 모델의 최종 비교. Decision Tree 는 GridSearchCV 의 취소 클래스(1) Precision·Recall·F1(0.7990 / 0.7513 / 0.7744) 및 전체 Accuracy·ROC-AUC(0.8377 / 0.8903)를 적용했다. 나머지는 팀 최종 비교표 기준이다. 개별 실행본의 분할·버전 차이로 완전 동일 조건 비교 여부는 미확인이다.

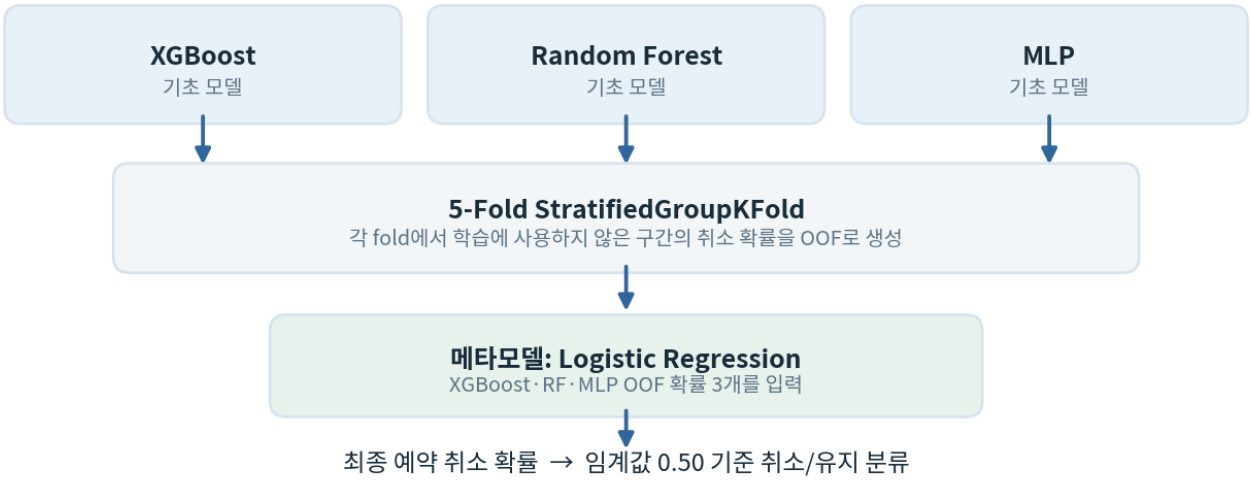




최종 선정 판단 Stacking 의 F1 0.8108, Accuracy 0.8658 은 전체 후보 중 최고이며 Recall 0.7755 도 XGBoost(0.7685)보다 높다. 단, ROC-AUC 최고는 XGBoost 0.9407 로 Stacking 모델보다 0.0005 높다.

04 최종 모델 Stacking 모델과 OOF 학습 구조

비교를 통해 선정한 Stacking 의 실제 내부 학습 절차를 상세히 살펴본다.



4.1 Out-of-Fold 학습 단계

- ① Stacking 의 내부 Train 에서 StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)를 구성한다.
- ② 각 기초 모델을 4 개 fold 로 학습하고, 학습에 사용되지 않은 나머지 1 개 fold 의 취소 확률을 예측한다.
- ③ 이 과정을 5 회 반복하여 학습 표본마다 XGBoost · Random Forest · MLP 의 OOF 확률을 하나씩 확보한다.
- ④ OOF 예측 3 열을 입력으로 Logistic Regression 메타모델(max_iter=2000)을 학습한다.
- ⑤ 각각의 기초 모델을 내부 Train 전체로 다시 fit 하여 Validation/Test 예측 확률을 얻고, 이를 메타모델에 입력한다.

4.2 왜 OOF 방식이 필요한가?

기초 모델이 자신이 학습했던 같은 표본에 대해 계산한 확률로 메타모델을 학습시키면, 메타모델은 지나치게 낙관적인 예측값에 의존할 수 있다. OOF 는 기초 모델의 학습에 포함되지 않았던 fold 에서 확률을 얻기 때문에 이 문제를 완화한다. 코드상 OOF 결측 예측값은 0 건이었고, 내부 Train/Validation 및 Test 사이 group_id 교집합도 0 건으로 확인됐다.

4.3 확률에서 취소 여부로 변환

세 기초 모델의 예측 확률 → 메타모델의 최종 취소 확률 → 기본 Threshold 0.50 이상이면 취소(1)로 분류한다. 메타모델 자체의 하이퍼파라미터나 Threshold 를 추가 탐색한 기록은 업로드된 StackingB.ipynb 에 없으므로 최적화가 확인된 범위는 기초 모델에 한정한다.

05 최종 Stacking 의 하이퍼파라미터 튜닝

기초 모델별 20 개 조합 × 그룹 5-Fold = 모델당 100 회, 총 300 회 교차검증 학습

5.1 탐색 공통 설정

RandomizedSearchCV(n_iter=20, scoring="roc_auc", cv=StratifiedGroupKFold(5), random_state=42, refit=True)를 사용하였다. 데이터 누수를 줄이기 위해 인코딩 및 스케일링을 각 기초 모델 Pipeline 에 포함했고, 그룹 정보(groups_train)를 검색 과정에 전달했다.

기초 모델	탐색한 주요 파라미터	최고 CV ROC-AUC
XGBoost	트리 수, 깊이, 학습률, min_child_weight, 행·열 샘플링, L2 규제	0.925171
Random Forest	트리 수, 깊이, 최소 분할 표본, 최소 리프 표본, 특성 샘플링	0.928611
MLP	은닉층 구성, alpha, 초기 학습률, batch_size	0.917791

5.2 선택된 최적 설정

모델	선택된 하이퍼파라미터
XGBoost	n_estimators=500; max_depth=5; learning_rate=0.1; min_child_weight=5; subsample=1.0; colsample_bytree=1.0; reg_lambda=1
Random Forest	n_estimators=500; max_depth=20; min_samples_leaf=3; min_samples_split=2; max_features=0.5
MLP	hidden_layer_sizes=(32,); alpha=0.01; learning_rate_init=0.001; batch_size=128

5.3 과적합 완화 관점의 튜닝 의미

- XGBoost 는 최대 깊이·min_child_weight·L2 규제를 활용해 트리 복잡도와 가중치 크기를 제어했다.
- Random Forest 는 최대 깊이와 최소 리프 샘플 수를 제한하여 특정 예약에 과도하게 맞춘 분할을 방지하고자 했다.
- MLP 는 L2 가중치 패널티(alpha=0.01)와 초기 정의의 early_stopping=True 를 통해 불필요한 복잡도와 과도한 반복 학습을 관리했다.

5.4 범주형 인코딩 빈도 기준 추가 실험

추가 실험 메모(notion.pdf)는 min_frequency=20 과 100 을 비교하였다. 당시 Validation 에서 20 의 ROC-AUC/F1 은 0.9402/0.8059, 100 은 0.9397/0.8042 로 20 이 소폭 우수했다. 다만 StackingB.ipynb 저장 실행본은 100 을 사용하므로 두 결과를 동일 학습 실행으로 합쳐 해석하지 않았다.

근거 범위 기초 모델 3 종의 튜닝 수행과 최적 CV 점수는 확인된다. 다만 Stacking 자체의 메타모델 또는 임계값을 별도로 튜닝했거나, 동일 조건의 Stacking 기본형 대비 튜닝 전후 성능 향상을 측정한 기록은 확인되지 않는다.

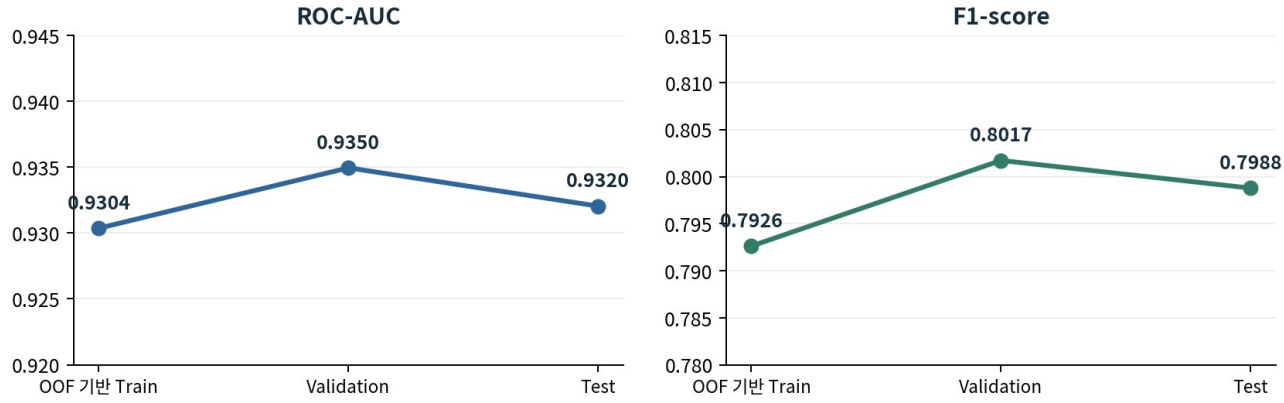
06 Stacking 과적합 여부 확인

교차검증과 OOF 학습을 적용한 후, 내부 Validation 및 Test 성능 저하 여부를 점검했다.

6.1 OOF 기반 Train · Validation · Test 비교

구간	Accuracy	F1-score	ROC-AUC	PR-AUC	Brier ↓
OOF 기반 Train	0.8528	0.7926	0.9304	0.9017	0.1034
Validation	0.8572	0.8017	0.9350	0.9087	0.0996
Test	0.8566	0.7988	0.9320	0.9038	0.1023

표 2. StackingB.ipynb 에 저장된 실행 결과. 팀 최종 비교표(Accuracy 0.8658, F1 0.8108 등)와는 다른 실행 버전이다.



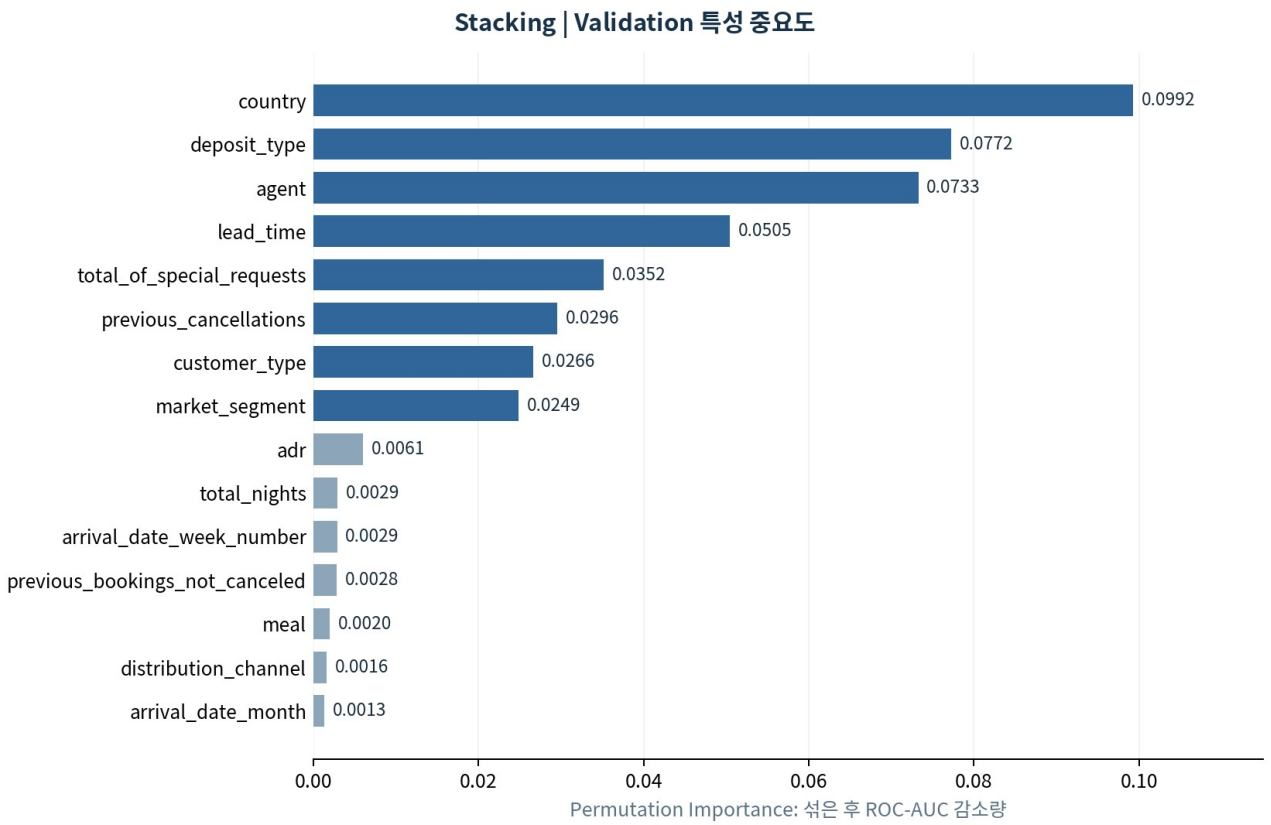
6.2 과적합 판단

- Validation 과 Test 의 ROC-AUC 차이는 약 0.0029, F1 차이는 약 0.0029 로 작아, 해당 실행본에서는 테스트 성능의 급격한 하락이 관찰되지 않았다.
- OOF 기반 Train ROC-AUC 0.9304 와 Validation ROC-AUC 0.9350 도 비슷하지만, Train OOF 확률로 메타모델을 학습했기 때문에 이를 엄밀한 독립 Train-Validation 격차로 볼 수는 없다.
- 그룹 5-Fold, 기초 모델 OOF 예측, Pipeline 내부 전처리, 모델 복잡도 규제 및 분리된 Test 평가를 통해 과적합 위험을 줄이도록 설계했다.

과적합 결론 이 실행 결과는 과적합 징후가 크지 않음을 뒷받침하지만, 과적합이 완전히 제거되었다고 단정할 수는 없다. 최종 선정 버전에 대해서도 동일 코드로 Train/Validation/Test 지표를 재확인할 필요가 있다.

07 Stacking 특성 중요도 분석

검증 데이터에서 각 입력 특성을 섞어 ROC-AUC 가 감소한 정도를 측정했다.



Permutation Importance / 검증 자료 / 특성별 무작위 섞기 3 회 평균 / StackingB.ipynb 저장 실행본

7.1 상위 변수와 의미

가장 큰 감소량은 country(0.0992), deposit_type(0.0772), agent(0.0733), lead_time(0.0505), total_of_special_requests(0.0352) 순서였다. 이 변수들이 해당 검증 자료에서 Stacking 의 예측 구분에 상대적으로 크게 기여했다는 의미다.

7.2 해석상 주의사항

Permutation Importance 는 특성값을 무작위로 섞었을 때 모델 성능이 얼마나 나빠지는지를 측정한다. 중요도가 높다고 예약 취소의 인과적 원인이라고 볼 수 없고, country · agent 는 데이터 표본 구성 또는 예약 채널 특성의 영향을 받을 수 있으므로 실무 적용 시 편향 · 분포 변화에 유의해야 한다. 낮은 중요도만으로 특성을 즉시 제거해서도 안 된다.

08 최종 모델 선정 근거의 세부 해석

Stacking 모델을 선택하되 XGBoost 와의 격차가 작다는 사실을 함께 제시한다.

8.1 가장 경쟁력 있는 XGBoost 와 직접 비교

지표	XGBoost	Stacking	Stacking − XGBoost
Accuracy	0.8644	0.8658	+0.0014
Precision	0.8514	0.8493	−0.0021
Recall	0.7685	0.7755	+0.0070
F1-score	0.8078	0.8108	+0.0030
ROC-AUC	0.9407	0.9402	−0.0005

F1-score 는 취소 예측의 정밀도와 재현율을 함께 평가한다. 팀 최종 비교표에서 Stacking 모델은 XGBoost 보다 Precision 이 0.0021 낮지만 Recall 이 0.0070, F1 이 0.0030 높아 프로젝트의 취소 포착과 예측 균형 목표에 따라 선택되었다. ROC-AUC 는 XGBoost 가 0.0005 앞섰다. 다만 저장된 실행본 간 평가 행·버전의 완전한 일치가 확인되지 않아 이 작은 차이를 일반화 성능의 확정적 우위로 해석하지 않는다.

8.2 Stacking 내부 Validation 에서도 비교

후보	ROC-AUC	F1	Brier Score ↓
XGBoost	0.9298	0.7930	0.1013
Random Forest	0.9338	0.7996	0.0984
MLP	0.9224	0.7862	0.1070
Stacking	0.9350	0.8017	0.0996

동일 Stacking 실험의 Validation 내에서는 결합 모델이 세 기초 모델보다 ROC-AUC 와 F1 에서 소폭 높았다. 다만 확률 예측 오차인 Brier Score 는 Random Forest 가 더 낮았다. 즉, Stacking 선택 이유는 성능 지표의 균형이지 모든 지표에서의 우위가 아니다.

8.3 기대 효과와 비용

- 장점: 서로 다른 모델의 예측 확률을 결합해 F1 과 Recall 을 소폭 개선했다.
- 고려 사항: 세 기초 모델과 메타모델을 함께 보관·실행해야 하므로 단일 XGBoost 보다 학습·배포·유지보수가 복잡해진다.
- 추가 검증: 성능 격차가 작으므로 반복 시드·시간 기준 테스트·신뢰구간을 확보해 일반화 우위를 확인하는 것이 타당하다.

09 임계값(Threshold)과 예측 오류 분석

Stacking 의 운영 관점 성능을 혼동행렬과 취소 누락(FN)·오경보(FP)로 구분해 해석한다. 아래 건수는 StackingB.ipynb 의 저장 실행본(테스트 23,842 건) 기준이며 팀 최종 비교표의 다른 실행 버전과 섞어 쓰지 않는다.

9.1 Threshold 0.50 기준 혼동행렬

원본 노트북에는 혼동행렬을 직접 출력한 셀은 없지만, 저장된 테스트 표본 수(23,842 건)와 Accuracy·Precision·Recall 의 정밀 출력값으로 네 칸의 정수 건수를 일관되게 역산할 수 있다.

실제 / 모델 예측	예측 유지(0)	예측 취소(1)	실제 합계
실제 유지(0)	TN = 13,637	FP = 1,365	15,002
실제 취소(1)	FN = 2,054	TP = 6,786	8,840
예측 합계	15,691	8,151	23,842

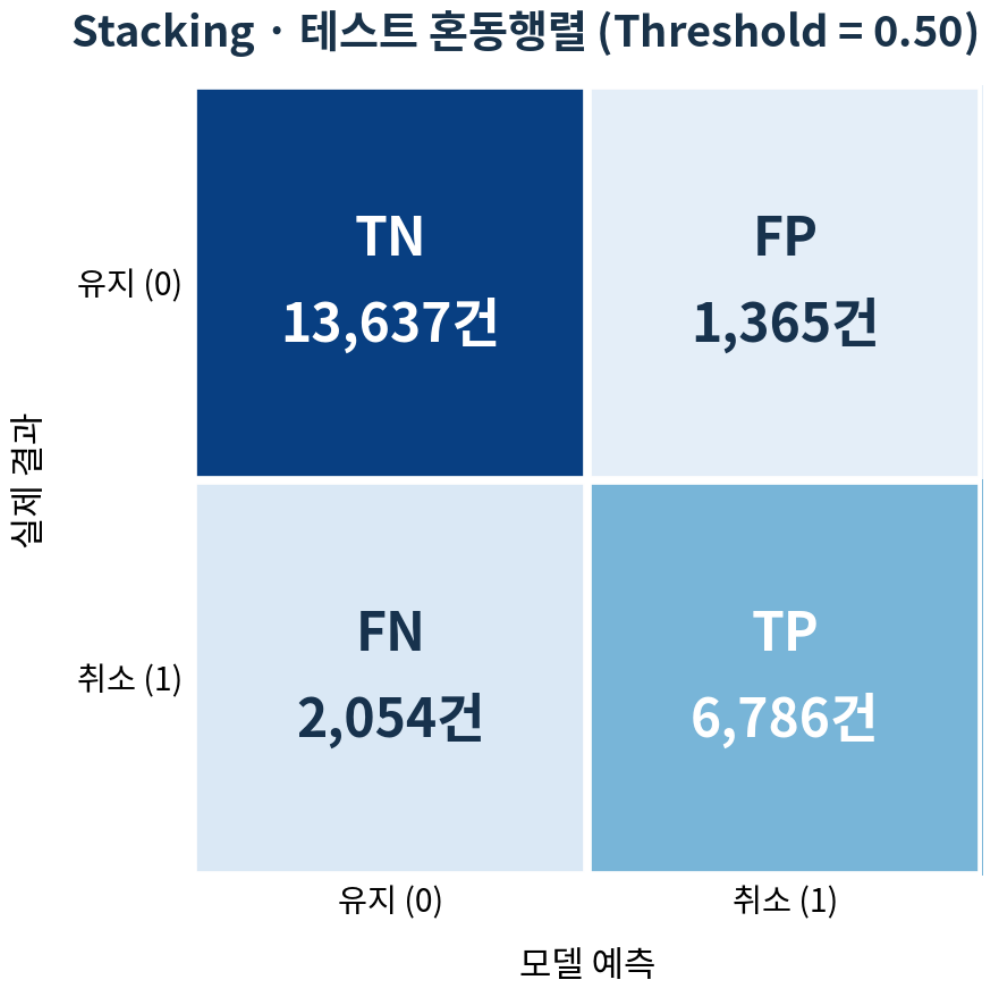


그림 6. StackingB.ipynb 테스트 지표로 역산한 혼동행렬. 0=예약 유지, 1=예약 취소. 팀 최종 모델 비교표의 수치와 다른 실행 기록임.

9.2 취소 탐지 정확도 및 오류 비용의 해석

지표	산식	결과	운영상 의미
Accuracy	(TP+TN) / 전체	0.8566	전체 예약 중 예측 정답 비율
Precision	TP / (TP+FP)	0.8325	취소 경고 8,151 건 중 실제 취소 비율
Recall	TP / (TP+FN)	0.7676	실제 취소 8,840 건 중 포착 비율
F1-score	2PR / (P+R)	0.7988	정밀도와 재현율의 조화평균
False Negative Rate	FN / (TP+FN)	23.24%	실제 취소 예약을 놓친 비율
False Positive Rate	FP / (TN+FP)	9.10%	실제 유지 예약을 잘못 경고한 비율

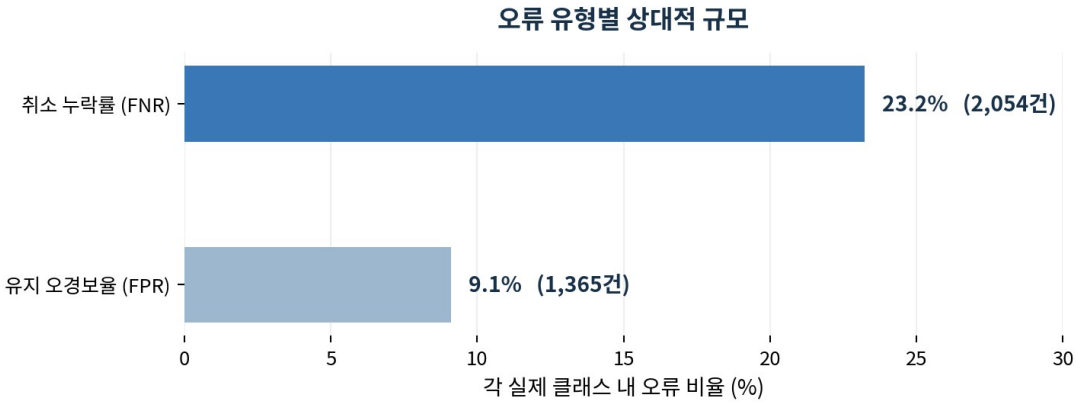


그림 7. FN 과 FP 는 모집단이 달라 건수만으로 비교하기 어렵다. 각각 실제 취소/유지 예약 대비 비율을 함께 확인했다.

미탐지 취소(FN)는 실제 취소될 예약을 위험도가 낮다고 판단하는 경우로, 사전 대응 기회를 놓칠 수 있다. 오탐(FP)은 유지될 예약에 불필요한 안내나 우대 혜택을 제공하는 비용을 유발할 수 있다. 실제 금전 손익은 호텔 운영 정책·고객 대응 비용·효과 데이터가 있어야 산정할 수 있다.

9.3 임계값 변화에 따른 Precision–Recall 관계

운영 시나리오	임계값 방향	일반적 예상 변화	주의사항
취소 누락 최소화	0.50 보다 낮춤	Recall ↑, FN ↓ / FP ↑ 가능	경고 대상 수·비용 증가 검토
현재 저장 실행본	0.50	F1=0.7988, Recall=0.7676	실측(저장 결과)의 기준점
불필요한 대응 축소	0.50 보다 높임	Precision ↑, FP ↓ / FN ↑ 가능	취소 고객을 더 많이 놓칠 수 있음

위 표의 0.50 이외 임계값은 실제 계산 결과가 아니라 일반적인 변화 방향이다. Stacking 의 확률 예측에 대해 임계값별 수치를 추가 실행해 확인해야 하며, 0.50 을 최적값으로 튜닝했다고 기술해서는 안 된다.

9.4 실제 운영 기준 선정 절차

① 최종 테스트 데이터를 제외한 검증용 확률을 확보한다. ② 예: 0.20~0.80 범위에서 Threshold 별 Precision·Recall·F1, FP·FN, 경고 대상 수를 계산한다. ③ 대응 한도와 취소 손실, 오경보 대응 비용을 반영해 기준을 고른다. ④ 선택한 기준을 고정한 뒤 오염되지 않은 최종 Test 에서 한 번 평가한다. 현재 노트북의 Validation 은 모델 비교에도 사용되었으므로 임계값 탐색을 추가할 경우 별도 평가 절차에 대한 주의가 필요하다.

10 예약 특성별 오류·세그먼트 분석

최종 모델의 전체 성능이 높더라도 특정 예약 유형에서 FN 또는 FP 가 집중될 수 있다. 현재 제공된 StackingB.ipynb 에는 그룹별 실제 오류 집계가 저장되어 있지 않으므로, 아래는 실행 가능한 분석 설계이며 관측 결과를 추정하여 채우지 않았다.

10.1 분석 대상 세그먼트와 분석 목적

세그먼트	구분 예시	분석 질문	현 상태
호텔 유형	City Hotel / Resort Hotel	유형별 취소 누락률 차이?	미산출
예약 리드타임	0–7 / 8–30 / 31–90 / 91 일 이상	예약까지 남은 기간별 FN 집중?	미산출
보증금 유형	No Deposit / Non Refund / 기타	보증 조건별 FP·FN 차이?	미산출
고객 유형	customer_type 별	특정 고객 유형에 높은 오경보?	미산출
국가·여행사 코드	country / agent	중요도 상위 변수의 편향·안정성?	미산출

10.2 공통 측정 지표 및 비교 원칙

각 집단의 샘플 수(n), 실제 취소 비중, TP·TN·FP·FN, Precision, Recall, F1-score, FNR=FN/(TP+FN), FPR=FP/(TN+FP)를 함께 집계한다. 단순 FN 건수는 집단 크기에 좌우되므로 집단 내 실제 취소 수로 나눈 FNR 을 반드시 병행한다. 분모가 0 인 경우에는 0 으로 기록하지 않고 '계산 불가'로 구분한다.

분석 축	표본 수	실제 취소율	FN	FP	FNR	FPR
City Hotel	원자료 필요	—	—	—	—	—
Resort Hotel	원자료 필요	—	—	—	—	—
lead_time 0~7 일	원자료 필요	—	—	—	—	—
lead_time 8~30 일	원자료 필요	—	—	—	—	—
lead_time 31~90 일	원자료 필요	—	—	—	—	—
lead_time 91 일 이상	원자료 필요	—	—	—	—	—

표의 대시(—)는 해당 데이터를 실제 계산하지 않았음을 뜻한다. 이는 분석 항목 누락이 아니라 관측되지 않은 수치를 임의로 제시하지 않기 위한 표시이다.

10.3 재현 가능한 세그먼트별 오류 집계 예시

아래 코드는 동일한 Stacking 테스트 데이터(X_test, y_test)와 예측 결과(stacking_test_pred)가 준비된 기존 노트북 환경에서 실행할 수 있도록 작성한 예시이다. 보고서 생성 과정에서는 원본 CSV 와 학습 모델 아티팩트가 함께 제공되지 않아 실행하지 않았다.

```
diag = X_test[["hotel", "deposit_type", "lead_time"]].copy()
diag["actual"] = y_test.to_numpy()
diag["pred"] = stacking_test_pred
diag["lead_group"] = pd.cut(diag["lead_time"],
  [-1, 7, 30, 90, np.inf], labels=["0-7", "8-30", "31-90", "91+"])
def summarize(g):
  a, p = g["actual"], g["pred"]
  tp = ((a==1)&(p==1)).sum(); fn = ((a==1)&(p==0)).sum()
  tn = ((a==0)&(p==0)).sum(); fp = ((a==0)&(p==1)).sum()
  return pd.Series({"n":len(g), "FN":fn, "FP":fp,
    "FNR":fn/(tp+fn) if tp+fn else np.nan,
    "FPR":fp/(tn+fp) if tn+fp else np.nan})
for field in ["hotel", "lead_group", "deposit_type"]:
  print(diag.groupby(field, observed=True).apply(summarize))
```

10.4 결과 해석 및 후속 대응 원칙

집단별 FNR 이 높다는 실제 결과가 확보된다면 해당 그룹에서 취소 위험을 놓치는 사례가 많다는 뜻이므로, 특성 분포·표본 수·변수 누수 여부를 확인하고 임계값·추가 학습을 검토할 수 있다. FP 가 집중된다면 불필요한 고객 안내 비용을 비교한다. 다만 지역·국가·여행사 코드의 중요도가 높다는 사실만으로 해당 집단을 취소 위험군으로 단정하거나 차별적인 정책을 적용해서는 안 된다.

현재 확보된 결론은 “세그먼트별 집계 항목과 방법을 설계했다”는 것까지이며, 특정 호텔 또는 예약 유형의 성능이 더 좋거나 나쁘다는 결과는 아직 확인되지 않았다.

11 저장 모델과 재현성 검증

노트북에 기록된 저장 단계와, 독립 실행 환경에서 아직 확인되지 않은 재현성 테스트를 구분해 최종 모델의 배포 준비도를 평가한다.

11.1 저장된 아티팩트 구조 및 확인된 사실

항목	노트북에서 확인된 내용	판정
저장 파일명	best_stacking_modelB2.joblib	저장 코드/출력 확인
저장 함수	joblib.dump(stacking_model, 경로)	실행 출력에 파일명 확인
기초 모델	fitted_models: XGBoost · Random Forest · MLP	3 개 Pipeline 포함
메타모델	LogisticRegression(max_iter=2000)	학습된 모델 포함
전처리	각 기초 모델 Pipeline 에 인코더/스케일러 포함	Pipeline 구성 확인
분류 임계값	노트북 평가 단계에서 0.50 적용	아티팩트 메타데이터에 별도 기록 없음
입력 스키마 및 모델 버전	별도 schema.json·metadata.json 저장 기록 없음	보완 필요
독립 프로세스 재로딩/재예측	실행 출력 없음	미검증

11.2 재로딩 및 신규 예약 예측 검증 코드 예시

아래 예시는 동일한 Python 환경에서 joblib 파일이 존재하고, 원래 25 개 입력 컬럼을 갖춘 X_new DataFrame 이 준비되었다는 전제에서 실행할 수 있다. 본 결과서에서는 이 코드를 새 프로세스에서 실행한 결과를 확보하지 못했다.

```
import joblib
import numpy as np
import pandas as pd

bundle = joblib.load("best_stacking_modelB2.joblib")
meta_x = pd.DataFrame({
    name: pipeline.predict_proba(X_new)[:, 1]
    for name, pipeline in bundle["base_models"].items()
}, index=X_new.index)
proba = bundle["meta_model"].predict_proba(meta_x)[:, 1]
pred = (proba >= 0.50).astype(int)
print(pd.DataFrame({"취소확률": proba, "취소예측": pred}))
```

메타모델은 기초 모델 예측 확률 세 열을 입력으로 학습했으므로, XGBoost·Random Forest·MLP 의 열 이름 및 순서를 학습 시점과 동일하게 유지해야 한다. 실제 서비스 입력은 원래 Train 에 사용된 동일 컬럼명·자료형을 충족해야 하며, 단순히 숫자를 세 개 넣는 것으로 예측이 성립하지 않는다.

11.3 배포 전 필수 재현성 점검 목록

검증 항목	확인 기준	현 단계
저장 파일 존재/해시	제출 디렉터리에 파일·SHA-256 기록	경로 실물 확인 필요
다른 프로세스에서 load	joblib.load 오류 없이 성공	미검증
동일 테스트 재예측	원본 테스트 점수/확률과 재로딩 결과 일치	미검증
단일 신규 예약 예측	한 행 입력으로 취소확률 반환	미검증
미등록 범주 처리	OneHotEncoder(handle_unknown="ignore") 동작 확인	설정 확인, 실행 미검증
필수 컬럼 누락/자료형	누락 거부·오류 안내 일관성 확인	미검증
스키마·라이브러리 버전	입력 컬럼 순서 및 패키지 버전 저장	미구현/미검증
서빙 환경	Streamlit 재학습 없이 load 후 predict	동작 검증 필요

11.4 기록 간 일치성·재현성의 한계

팀 최종 비교표의 Stacking 성능(Accuracy 0.8658, F1 0.8108, ROC-AUC 0.9402)과 StackingB.ipynb 에 저장된 Test 성능(Accuracy 0.8566, F1 0.7988, ROC-AUC 0.9320)은 서로 다른 실행 기록이다. 전자는 팀의 선정 근거로만 사용하고, 후자의 혼동행렬·과적합 검토와 혼합하지 않았다. 또한 팀 집계 Train/Test 95,367/23,841 건과 Stacking 저장 출력 95,366/23,842 건이 1 건씩 다르며, Random Forest 는 RAW 원본에서 자체 전처리·그룹 분할을 생성했다. 현재 자료만으로는 동일한 평가 행과 저장된 모델 버전의 완전 일치 또는 최종 성능 재현을 확정할 수 없다.

재현을 위해서는 최종 전처리 파일과 split, 전체 평가 행의 source_row_id 목록, 선택 모델의 저장 버전을 하나로 확정해야 한다. 같은 고정 테스트 집합에서 5 개 모델의 지표를 다시 산출한 뒤 모델 joblib, 입력 스키마, 라이브러리 버전, Threshold, 평가 결과를 함께 보관하는 것이 바람직하다.

12 최종 결론과 학습 결과 요약

다섯 후보를 사전 설계하여 비교한 후 Stacking 모델을 최종 모델로 결정했다.

12.1 핵심 질문 ① 과적합 여부를 확인하고 완화 방법을 적용했는가?

예. Stacking 에서는 StratifiedGroupKFold 5-Fold 기반 OOF 확률을 생성하고, 각 기초 모델의 학습에 사용되지 않은 fold 예측값으로 메타모델을 학습했다. 또한 XGBoost·Random Forest 의 트리 복잡도 제어, MLP 의 규제와 Early Stopping, Pipeline 내부 전처리 및 분리된 Validation/Test 평가를 적용하였다. 노트북 실행본의 Validation/Test ROC-AUC 는 0.9350/0.9320, F1 은 0.8017/0.7988 로 큰 성능 감소가 관찰되지 않았다. 다만 OOF 기반 Train 수치만으로 완전한 과적합 부재를 증명할 수는 없다.

12.2 핵심 질문 ② 하이퍼파라미터 튜닝으로 최적화를 수행했는가?

예. Stacking 의 XGBoost·Random Forest·MLP 각각에서 RandomizedSearchCV 20 회 및 StratifiedGroupKFold 5-Fold 를 실시하여 총 300 회의 교차검증 적합을 수행했다. 최고 CV ROC-AUC 는 각 0.9252, 0.9286, 0.9178 이었다. 기초 모델 최적 조합을 채택한 뒤 OOF 예측을 생성하고 Logistic Regression 메타모델을 학습하였다. 메타모델 파라미터와 Threshold 별도 튜닝의 실행 근거는 없다.

추가 분석으로 Stacking 저장 실행본의 혼동행렬(TN 13,637 / FP 1,365 / FN 2,054 / TP 6,786)을 도출했다. 모델은 저장 코드를 실행했으나 재로딩·신규 입력·스키마 검사까지 완료했다고 볼 수 없으며, 세그먼트별 오류는 추가 집계가 필요한 상태다.

12.3 결론 및 주요 후속 과제

최종 판단 최종 팀 비교표상 Stacking: Accuracy 0.8658 / Precision 0.8493 / Recall 0.7755 / F1-score 0.8108 / ROC-AUC 0.9402. 다섯 후보 가운데 F1 과 Accuracy 가 가장 높아 최종 모델로 선정하였다.

- **재현성: 팀 비교표와 StackingB.ipynb 의 성능 차이(Accuracy 0.8658 vs 0.8566, F1 0.8108 vs 0.7988, ROC-AUC 0.9402 vs 0.9320), 데이터 1 건 차이, Random Forest 의 자체 데이터 분할을 최종 고정 평가 집합에서 재확인한다.**

- 모델 안정성: 반복 시드, 시간 기준 Holdout, 호텔 유형별/국적별 성능 및 확률 보정 상태를 추가 점검한다.
- 운영: 비용에 맞는 Threshold 선택과 모델 유지보수 비용을 함께 검토한다.

12.4 참고 자료

DecisionTree.ipynb · logistic_regression.ipynb · Random Forest - 오원아.ipynb · XGBoost 분류 모델.ipynb · StackingB.ipynb · 개인별 모델 학습 결과서 4 종 · notion.pdf · 팀 최종 5 모델 비교표. 이 문서는 저장된 기록을 교차 점검했으며, 원본 데이터/모델을 재학습해 성능을 재산출하지는 않았다.